



كلية تكنولوجيا الصناعة والطاقة

Information Technology Department



جامعة برج العرب التكنولوجية
BORG AL ARAB TECHNOLOGICAL UNIVERSITY

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year – Information Technology Program

spring 2026

COURSE INFORMATION

- Lecturer: Dr. Dina Abdelhafiz, Dr. Ghada Fathy
- Credit hours (3)
- Requirements & Grading (Total 150 marks)
 - Class work and attendance (40 marks)
 - Midterm exam (35 marks)
 - Final Exam during finals week (75 marks)
 - Final Exam in hours (3 hours)



LECTURE 2

C LANGUAGE

BASICS

DATA TYPES, VARIABLES, INPUT, OUTPUT



STRUCTURE OF A C PROGRAM

Part	Purpose	Example
Preprocessor	Import libraries	<code>#include <stdio.h></code>
Main function	Program start	<code>int main()</code>
Variables	Store data	<code>int x= 5;</code>
Statements	Program logic	<code>printf();</code>
Return	End program	<code>Return 0;</code>

DATA TYPES OR VARIABLES IN C

- Variables are containers for storing data values, like numbers and characters.
- You can think of a variable as a named box where you keep a value that can be used later.

Type	Meaning	Example	Output
int	Whole numbers, such as 123 or -123	int a=10;	10
float	Decimal, such as 19.99 or -19.99	float x=3.5;	3.500000
double	Sufficient for storing 15 decimal digits	double x= 3.55555;	1.99
char	Character, such as 'a' or 'B'	char g='A';	A

C THE SIZEOF OPERATOR

- The memory size refers to how much space a type occupies in the computer's memory.
- To actually get the size (in bytes) of a data type or variable, use the sizeof operator:

Data Type	Size
int	2 or 4 bytes
float	4 bytes
double	8 bytes
char	1 byte

```
#include <stdio.h>

int main() {
    int myInt;
    float myFloat;
    double myDouble;
    char myChar;

    printf("%zu\n", sizeof(myInt));
    printf("%zu\n", sizeof(myFloat));
    printf("%zu\n", sizeof(myDouble));
    printf("%zu\n", sizeof(myChar));

    return 0;
}
```

DECLARING (CREATING) VARIABLES

- `// Declare a variable`
`int myNum; /* int is a data type, and variableName is the name you choose (for example myNum).*/`

`// Assign a value later`
`myNum = 15;`
- `// The equal sign = is used to assign a value to the variable`
- **Tip:** Variable names should be meaningful, so your code is easier to understand.

EXAMPLE

- Create a variable named **age** of type **int** and assign the value **21**:

```
int age =21;
```

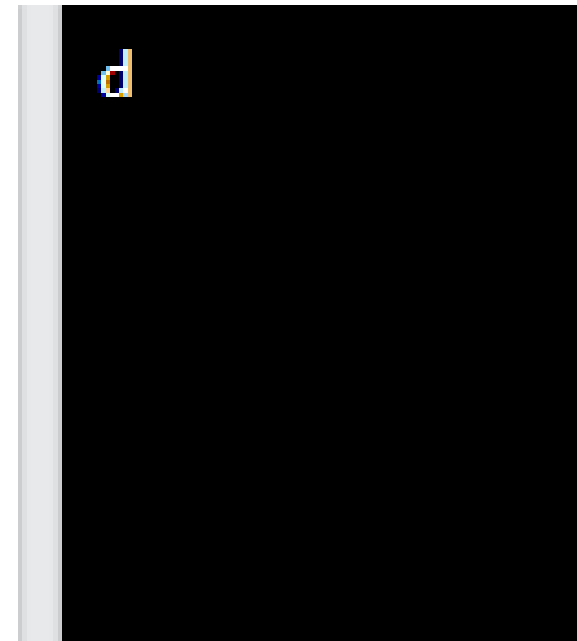
// Note: Variable names should be meaningful, so your code is easier to understand

NOTES ON CHARACTERS

- If you try to store more than a single character, it will only print the last character:

```
#include <stdio.h>

int main() {
    char myText = 'HelloWorld';
    printf("%c", myText);
    return 0;
}
```



C FORMAT SPECIFIERS

- Format specifiers are used together with the `printf()` function to print variables.
- You can think of a format specifier as a **placeholder** that tells C what kind of value will be printed and where.
- A format specifier always starts with a percentage sign `%`, followed by a letter.

Specifier	Used For	Example	Output
<code>%d</code>	Integer	<code>printf("%d",5);</code>	5
<code>%f</code>	Float	<code>printf("%f",3.2);</code> <code>printf("%.2f",3.2);</code>	3.200000 3.20
<code>%lf</code>	Double (long float)	<code>printf("%.lf",3.2);</code>	3.200000
<code>%c</code>	Char	<code>printf("%c",'A');</code>	A
<code>%s</code>	String	<code>printf("%s","Hi");</code>	Hi

EXAMPLE

```
#include <stdio.h>

int main() {
    // Create variables
    int myNum = 15;           // Integer (whole number)
    float myFloatNum = 5.99; // Floating point number
    char myLetter = 'D';     // Character

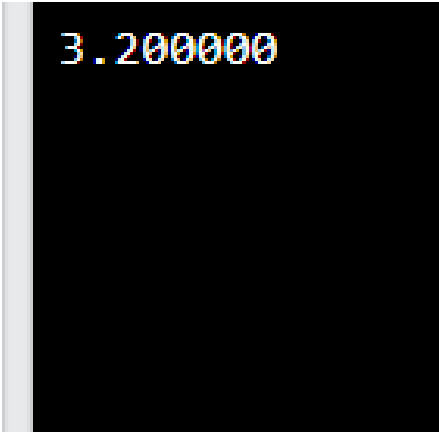
    // Print variables
    printf("%d\n", myNum);
    printf("%f\n", myFloatNum);
    printf("%c\n", myLetter);
    return 0;
}
```

```
15
5.990000
D
```

Note: Each format specifier must match the correct type of value.

EXAMPLE ON DOUBLE

```
#include <stdio.h>
int main() {
    printf("%lf", 3.2);
    return 0;
}
```

A terminal window with a black background and white text. The text displayed is "3.200000".

3.200000

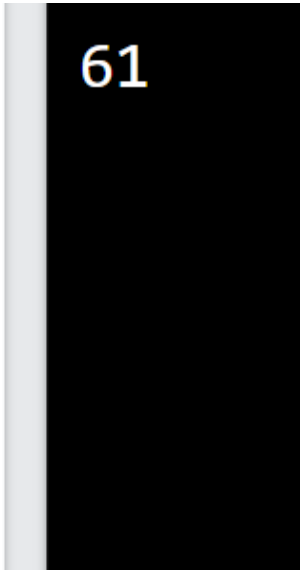
The precision of float is **six or seven decimal digits**, while double variables have a precision of about **15 digits**.

C DECLARE MULTIPLE VARIABLES

- **Note:** All variables must be of the same type. You cannot mix types in the same declaration.
- **Tip:** Declaring multiple variables on one line is convenient, but declaring them on separate lines can make your code easier to read.

```
#include <stdio.h>

int main() {
    int x = 5, y = 6, z = 50;
    printf("%d", x + y + z);
    return 0;
}
```

A terminal window with a black background and a light gray vertical scrollbar on the left. The number 61 is displayed in white text at the top of the window.

61

C VARIABLE NAMES (IDENTIFIERS)

- All C **variables** must be **identified** with **unique names**.
- These unique names are called **identifiers**.
- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

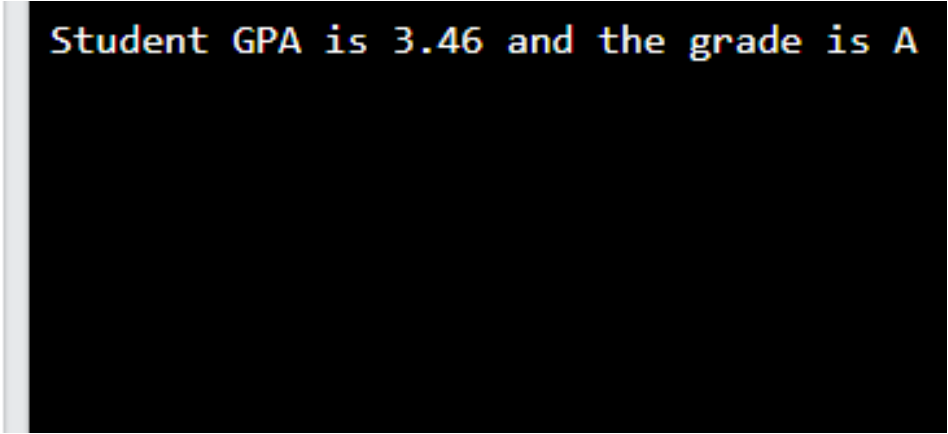
The **general rules** for naming variables are:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore ()
- Names are case-sensitive (myVar and myvar are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (such as int) cannot be used as names

PRINT MULTIPLE VARIABLES

■ Print student grade and GPA

```
#include <stdio.h>
int main() {
    char grade= 'A';
    float GPA= 3.456;
    printf("Student GPA is %.2f and the grade is %c", GPA, grade);
    return 0;
}
```



```
Student GPA is 3.46 and the grade is A
```

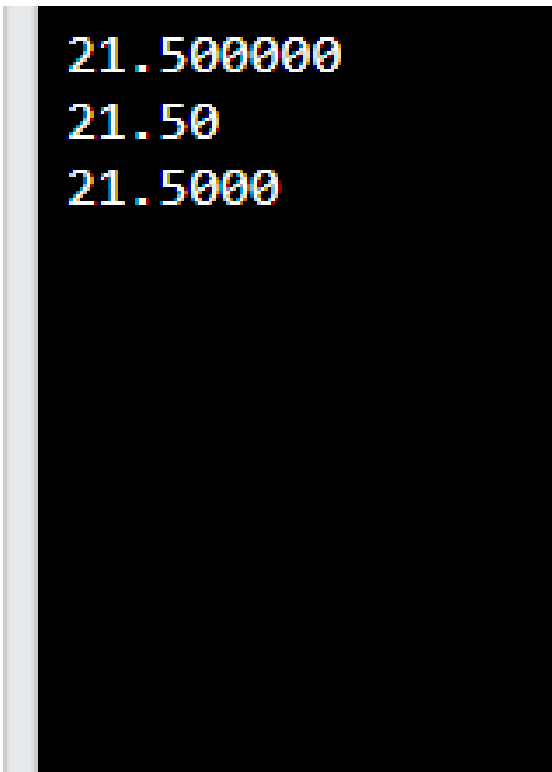
CHANGE VARIABLE VALUES

- If you assign a new value to an existing variable, the new value will **replace** the old one:

```
#include <stdio.h>

int main() {
    float myGrade = 15; // myNum is 15
    myGrade = 21.5; // Now myNum is 21.5

    printf("%f\n", myGrade); //21.500000
    printf("%.2f\n", myGrade); //21.50
    printf("%.4f\n", myGrade); //21.5000
    return 0;
}
```



```
21.500000
21.50
21.5000
```


C CONSTANTS

- the `const` keyword, which makes a variable **unchangeable** and **read-only**:

```
#include <stdio.h>

int main() {
    const int minutesPerHour = 60;
    const int monthsInYear = 12;

    printf("%d\n", minutesPerHour);
    printf("%d\n", monthsInYear);
    return 0;
}
```

```
60
12
```

When you declare a constant variable, it must be assigned with a value and donot change!

```
#include <stdio.h>

int main() {
    const int myNum = 15;
    myNum = 10;

    printf("%d", myNum);
    return 0;
}
```

```
prog.c: In function 'main':
prog.c:5:18: error: assignment of read-only variable 'myNum'
   5 |     myNum = 10;
     |           ^
```

C USER INPUT

- To get **user input**, you can use the `scanf()` function:

- Input (from user) variables: grade, GPA, age

then display/print/output to screen

```
#include <stdio.h>

int main() {
    // Create a string
    char grade;
    float GPA;
    int age;

    // Ask the user to input grade
    printf("Enter your grade: \n");
    scanf("%c", grade);

    printf("Enter your GPA: \n");
    scanf("%f", GPA);

    printf("Enter your age: \n");
    scanf("%d", age);

    // Output the text
    printf("My age is %d and my grade is %c with GPA %f", age, grade, GPA);

    return 0;
}
```

■ Create a Student Report

- Inside main(), complete the following steps:
- Declare an int named studentID and assign it a value
- Declare a float named score and assign it a value
- Declare a char named grade and assign it a single character (in single quotes)
- Print all three values using printf (use the correct format specifiers).

```
#include <stdio.h>

int main() {
    // Write studentID here
    // Write score here
    // Write grade here

    // Print studentID
    // Print score
    // Print grade

    return 0;
}
```

■ Create a Student Report

- Inside main(), complete the following steps:
- Declare an int named studentID and ask the user to input value
- Declare a float named score and ask the user to input value
- Declare a char named grade and ask the user to input value
- Print all three values using printf (use the correct format specifiers).

```
#include <stdio.h>

int main() {
    // Write studentID here
    // Write score here
    // Write grade here

    // Print studentID
    // Print score
    // Print grade

    return 0;
}
```



Happy Coding!